LKPD 9 - CI/CD Docker Swarm dengan GitHub Actions

Oct 12, 2026 · @nugteam1

Identitas LKPD

Komponen	Keterangan
Mata Pelajaran	DevOps Engineer
Topik	CI/CD Docker Swarm dengan GitHub Actions
Kelas / Semester	_____
Nama Siswa	_____
Kelompok	_____
Alokasi Waktu	5 x 45 menit
Lingkungan Praktik	AWS Academy Learner Lab, cluster swarm dari LKPD 8

Prasyarat

- Cluster swarm dari LKPD 8 masih berjalan, MariaDB aktif di instance `db` (container `db`, port 3306), dan appKantin pernah berhasil dijalankan.
- Folder `/home/ubuntu/appKantin/config/db.php` di node manager sudah berisi `DB_HOST = "<IP-PRIVATE-DB>"` dan password database.
- Memiliki akun GitHub dan akun Docker Hub (gratis).

Tujuan Pembelajaran

Setelah menyelesaikan LKPD ini, siswa mampu:

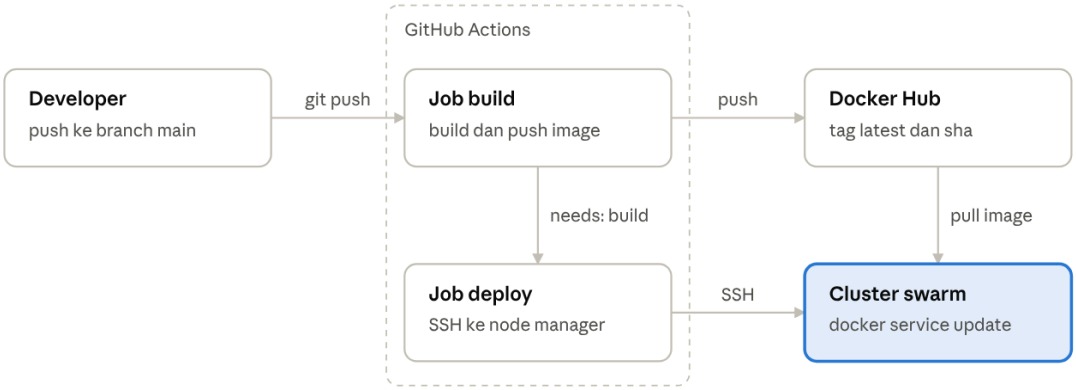
- Membuat Dockerfile yang mengemas kode aplikasi ke dalam image.
- Membuat workflow GitHub Actions yang membangun image dan mengirimnya ke Docker Hub setiap ada push.
- Memisahkan konfigurasi dari kode memakai Docker config.
- Mengotomatiskan rolling update service di swarm dari workflow, lalu melakukan rollback bila perlu.

Skenario

Setiap kali kode di branch `main` berubah, GitHub Actions membangun image baru, mengirimnya ke Docker Hub, lalu memerintahkan swarm memperbaruinya secara bertahap, tanpa Anda login ke server.

- **CI (continuous integration):** job `build` mengambil kode, membangun image dari `Dockerfile`, dan mengirimnya ke Docker Hub dengan tag `latest` dan tag sha commit.
- **CD (continuous delivery):** job `deploy` masuk ke node manager lewat SSH lalu menjalankan `docker service update` dengan image bertag sha tersebut. Swarm mengganti replika satu per satu, dan setiap node menarik image dari Docker Hub.
- **Konfigurasi terpisah dari kode:** `config/db.php` disimpan sebagai Docker config swarm dan dipasang ke container, sehingga password database tidak masuk repository maupun image.

Satu git push memicu build, push image, lalu rolling update di swarm



alur CI/CD · 5 tahap dari git push sampai rolling update

Kotak berwarna adalah cluster swarm dari LKPD 8, sedangkan kotak putus-putus adalah lingkungan GitHub Actions.

Persiapan

IP public manager berubah setiap Learner Lab di-restart. Jika Anda melanjutkan praktik di sesi lab baru, perbarui secret `SSH_HOST` (langkah E) dengan IP public yang baru.

A. Docker Hub: buat access token

1. Login ke hub.docker.com.
2. Buka **Account settings** → **Personal access tokens** → **Generate new token**.
3. Nama: `github-actions`, izin: **Read & Write**. Salin token (hanya ditampilkan sekali).

Repository `appkantin` akan terbentuk otomatis saat image pertama di-push.

B. GitHub: fork appKantin

Buka `https://github.com/paknux/appKantin`, klik **Fork**, lalu simpan di akun GitHub Anda. Semua perubahan pada LKPD ini dilakukan di fork milik Anda, bukan di repository asli. File dapat diedit lewat `git` di komputer sendiri atau lewat editor web GitHub (**Add file** → **Create new file**, atau ikon pensil pada file yang sudah ada).

C. Manager: buat kunci SSH untuk deploy

Workflow akan masuk ke manager lewat SSH untuk menjalankan `docker service update`. Buat sepasang kunci khusus untuk keperluan ini di node manager:

```
ssh-keygen -t ed25519 -f ~/.ssh/gha_deploy -N ""
cat ~/.ssh/gha_deploy.pub >> ~/.ssh/authorized_keys
cat ~/.ssh/gha_deploy
```

Perintah terakhir menampilkan kunci privat. Salin seluruh isinya, termasuk baris `BEGIN` dan `END`, untuk dipakai pada langkah E.

D. AWS: izinkan GitHub mengakses SSH manager

Alamat IP server GitHub Actions tidak tetap, sehingga rule SSH pada `sg-swarm` diubah sementara menjadi sumber `0.0.0.0/0`. Login tetap hanya bisa dengan kunci (bukan password). Kembalikan ke `My IP` pada tahap pembersihan.

E. GitHub: simpan secrets

Pada fork Anda: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.

Nama secret	Isi
`DOCKERHUB_USERNAME`	Username Docker Hub (huruf kecil)
`DOCKERHUB_TOKEN`	Token dari langkah A
`SSH_HOST`	IP public node manager
`SSH_PRIVATE_KEY`	Isi kunci privat dari langkah C

Secret tidak ditulis di file workflow. Nilainya disembunyikan di log dan hanya dibaca saat workflow berjalan.

Langkah Kerja

Langkah 1, 2, dan 5 dikerjakan pada repository fork di GitHub. Langkah 3, 4, 6, dan 7 dikerjakan di terminal node manager. Ganti `<DOCKERHUB_USERNAME>` dengan username Docker Hub Anda.

Langkah 1. Tambahkan Dockerfile

Di akar repository, buat file `Dockerfile`:

```
FROM trafex/php-nginx
COPY --chown=nobody . /var/www/html/
```

Image dasar `trafex/php-nginx` sudah berisi nginx dan PHP-FPM. Seluruh kode aplikasi disalin ke dalam image, sehingga tidak perlu lagi bind mount atau `git pull` di setiap node.

Buat juga file `.dockerignore` agar berkas yang tidak perlu tidak ikut masuk image:

```
.git
.github
README.md
```

Langkah 2. Workflow CI: build dan push image

Buat file `.github/workflows/deploy.yml`. Pada tahap ini baru ada satu job yang membangun image lalu mengirimnya ke Docker Hub.

```
name: CI-CD appKantin

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/appkantin:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/appkantin:${{ github.sha }}
```

Setiap build diberi dua tag: `latest` dan `github.sha` (kode unik commit). Tag sha menandai versi yang persis dan dipakai saat deploy.

Commit ke branch `main`, lalu buka tab **Actions** di GitHub. Tunggu hingga job `build` berwarna hijau, kemudian periksa Docker Hub: repository `appkantin` harus memiliki tag `latest` dan satu tag sha.

Catatan: nomor versi action (`@v4`, `@v3`, `@v6`) dapat bertambah seiring waktu. Jika GitHub menampilkan peringatan versi usang, naikkan ke versi mayor terbaru.

Langkah 3. Pisahkan konfigurasi dari kode [manager]

Kode sekarang ada di dalam image, sedangkan `config/db.php` berisi alamat dan password database yang tidak boleh masuk image atau repository. File itu disimpan sebagai **Docker config** milik swarm, lalu dipasang ke dalam container saat service dibuat.

Pastikan `/home/ubuntu/appKantin/config/db.php` di manager sudah berisi `DB_HOST = "<IP-PRIVATE-DB>"` dan password yang benar. Lalu hapus service lama dan buat config:

```
docker service rm kantin
docker config create kantin_db_php /home/ubuntu/appKantin/config/db.php
docker config ls
```

Langkah 4. Buat service dari image [manager]

```
docker service create --name kantin \
  --replicas 3 -p 8080:8080 \
  --config source=kantin_db_php,target=/var/www/html/config/db.php \
```

```
  <DOCKERHUB_USERNAME>/appkantin:latest

docker service ps kantin
```

Setiap node menarik image dari Docker Hub sendiri (repository bersifat publik). Buka `http://<IP-PUBLIC-MANAGER>:8080` dan pastikan aplikasi tampil dan terhubung ke database. Catat tag image yang sedang berjalan:

```
docker service inspect kantin --format '{{.Spec.TaskTemplate.ContainerSpec.Image}}'
```

Langkah 5. Workflow CD: tambahkan job deploy

Ganti isi `.github/workflows/deploy.yml` dengan versi lengkap berikut (job `build` tetap sama, ditambah job `deploy`):

```
name: CI-CD appKantin

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/appkantin:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/appkantin:${{ github.sha }}

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SSH_HOST }}
          username: ubuntu
          key: ${{ secrets.SSH_PRIVATE_KEY }}
          script: |
            docker service update \
              --image ${{ secrets.DOCKERHUB_USERNAME }}/appkantin:${{ github.sha }} \
              --update-parallelism 1 --update-delay 10s \
              kantin
```

`needs: build` membuat deploy hanya berjalan jika build berhasil. Job deploy masuk ke manager lewat SSH, lalu memperbarui service ke tag sha dari commit tersebut dengan rolling update, satu replika setiap 10 detik.

Langkah 6. Uji alur otomatis

1. Ubah tampilan aplikasi di fork, misalnya teks judul pada `header.php`. Commit ke `main`.
2. Di tab **Actions**, amati job `build` lalu `deploy` sampai keduanya hijau.
3. Di manager, amati rolling update saat berlangsung:

```
watch docker service ps kantin
```

Replika lama berstatus `Shutdown`, replika baru `Running` dengan tag sha yang baru.

4. Refresh browser. Perubahan harus tampil tanpa Anda menyentuh server sama sekali.

Langkah 7. Rollback

Jika versi baru bermasalah, kembalikan ke versi sebelumnya:

```
docker service rollback kantin
docker service inspect kantin --format '{{.Spec.TaskTemplate.ContainerSpec.Image}}'
```

Perhatikan bahwa push berikutnya ke `main` akan men-deploy versi terbaru lagi.

Tugas

Tugas 1. Rilis fitur kecil lewat pipeline

Tambahkan satu baris di `footer.php` pada fork Anda yang menampilkan hostname container, misalnya `<small>Dilayani oleh: <?= gethostname() ?></small>`. Commit ke `main`, tunggu job `deploy` selesai, lalu refresh browser berulang kali. Hostname harus berganti-ganti antar replika. Sertakan tangkapan layar tab Actions dan browser.

Tugas 2. Rilis versi rusak, lalu rollback

Sengaja buat kesalahan sintaks PHP pada salah satu berkas (misalnya hapus tanda `;` pada `header.php`), lalu commit ke `main`.

1. Amati apakah pipeline tetap hijau dan apakah aplikasi masih bisa dibuka.
2. Pulihkan aplikasi dengan `docker service rollback kantin`.
3. Perbaiki kode di repository dan push lagi sampai aplikasi normal.

Catat temuan Anda: mengapa pipeline sampai men-deploy kode yang rusak?

Tugas 3 (tantangan). Hentikan pipeline sebelum kode rusak sampai ke server

Tambahkan langkah pengecekan sintaks PHP pada job `build`, tepat sebelum langkah `docker/build-push-action`:

```
      - name: Cek sintaks PHP
        run: |
          for f in $(find . -name '*.php'); do
            php -l "$f" || exit 1
          done
```

Ulangi percobaan kode rusak dari Tugas 2. Kali ini job `build` harus gagal sehingga job `deploy` tidak berjalan dan aplikasi di server tetap normal. Jika `php` tidak ditemukan pada runner, tambahkan langkah `shivammathur/setup-php@v2` sebelum pengecekan.

Pertanyaan Refleksi

Jawab dalam laporan:

1. Mengapa image diberi tag `github.sha` dan tidak hanya `latest`? Apa yang sulit dilakukan jika hanya ada `latest`?
2. Bandingkan cara ini dengan bind mount + `git pull` terjadwal (LKPD 8). Sebutkan 2 keunggulan dan 1 kekurangan memakai image.
3. Mengapa `config/db.php` disimpan sebagai Docker config dan tidak dimasukkan ke image atau repository?
4. Apa risiko membuka port 22 untuk `0.0.0.0/0`? Bagaimana self-hosted runner pada node manager dapat menghindari kebutuhan ini?
5. Jelaskan mana yang disebut CI dan mana yang CD pada workflow ini.
6. Pada Tugas 2, mengapa swarm tidak otomatis rollback walaupun aplikasi rusak? Apa yang perlu ditambahkan agar kerusakan terdeteksi sebelum atau saat deploy?

Troubleshooting

Gejala	Penyebab umum	Solusi
Login Docker Hub gagal di job `build` (`unauthorized`)	Username/token salah, atau token tidak berizin Read & Write	Periksa secret `DOCKERHUB_USERNAME` dan `DOCKERHUB_TOKEN`; buat token baru
Push gagal (`denied: requested access`)	Nama repository tidak sesuai username, atau username memakai huruf besar	Gunakan username huruf kecil pada tag
Job `deploy` timeout saat SSH	Rule port 22 belum terbuka untuk GitHub, atau `SSH_HOST` sudah usang karena IP public berubah	Atur sumber SSH ke `0.0.0.0/0`; perbarui secret `SSH_HOST`
`Permission denied (publickey)`	Isi `SSH_PRIVATE_KEY` tidak lengkap, atau kunci publik belum ada di `authorized_keys` manager	Salin ulang seluruh kunci privat termasuk baris BEGIN/END; ulangi langkah C
`permission denied` pada `docker.sock` di job deploy	User `ubuntu` belum masuk grup `docker`	`sudo usermod -aG docker ubuntu`, lalu coba lagi
`service kantin not found`	Service belum dibuat atau namanya berbeda	Selesaikan Langkah 4; cek `docker service ls`
Replika `Rejected` atau gagal menarik image	Nama image salah, atau repository Docker Hub bersifat private	Periksa nama dan tag; jadikan repository publik

Gejala	Penyebab umum	Solusi
Aplikasi error koneksi database	`DB_HOST` di config bukan IP private instance `db`, container `db` mati, atau rule TCP 3306 belum dibuka	Di instance `db` jalankan `docker ps`; uji koneksi seperti Tugas 2 LKPD 8; bila isi config salah, buat config baru dengan nama lain

Catatan: Docker config tidak dapat diubah setelah dibuat. Untuk mengganti isinya, buat config baru dengan nama lain, lalu perbarui service dengan `--config-rm` dan `--config-add`.

Pembersihan

- Kembalikan sumber rule SSH pada `sg-swarm` menjadi `My IP`.
- Hapus access token di Docker Hub dan hapus kunci `gha_deploy` dari `~/.ssh/authorized_keys` di manager jika tidak dipakai lagi.
- Hapus secrets di repository fork bila praktik selesai.
- Klik **End Lab** di Learner Lab saat berhenti praktik.

Laporan dan Penilaian

Laporan dibuat dalam satu berkas dan dikumpulkan di GC. Isi laporan:

1. Identitas (nama, kelas, kelompok) dan tujuan praktik.
2. Tangkapan layar: isi `Dockerfile` dan `deploy.yml`, tab Actions dengan job hijau, tag image di Docker Hub, `docker service ps kantin` sebelum dan sesudah update, serta aplikasi di browser.
3. Hasil Tugas 1 sampai 3 (Tugas 3 bila dikerjakan) beserta catatan temuan.
4. Jawaban Pertanyaan Refleksi.
5. Kendala yang dialami dan cara mengatasinya, serta kesimpulan.

Aspek yang dinilai	Bobot
Dockerfile dan image berhasil terbit di Docker Hub	15
Workflow CI (build dan push) berjalan	20
Service berjalan dari image dengan Docker config	15
Workflow CD (deploy otomatis dan rolling update)	25
Tugas 2 dan rollback	10
Jawaban pertanyaan refleksi	10
Kelengkapan dan kerapian laporan	5
Total	**100**

Tugas 3 (tantangan) dapat dijadikan nilai tambahan.